Non-Linear Latent Variable Models & Variational Auto-Encoders (VAEs)

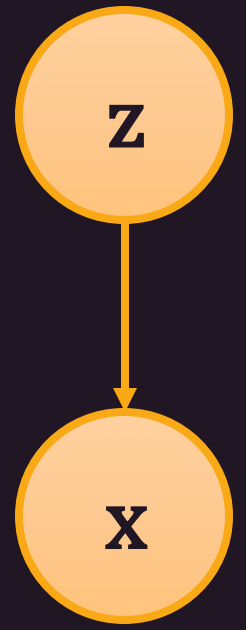
Chirag Raman
Assistant Professor
Department of Intelligent Systems, TUDelft

Generative Modeling DSAIT4030, 06/05/2025

Latent variables: the story so far

$$p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{x}|\mathbf{z})p(\mathbf{z})$$ e.g. Gaussian Mixture Model

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$$ e.g. Probabilistic PCA



Now if we want to fit this model to data with some parameter $\mathbf{w}$, we typically maximize the likelihood function

$$p(\mathbf{x}|\mathbf{w}) = \int p(\mathbf{x}|\mathbf{z}, \mathbf{w})p(\mathbf{z})d\mathbf{z}$$

For PPCA, we assumed $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})$

$$\mathbf{x} = \mathbf{W}\mathbf{z} + \boldsymbol{\mu} + \boldsymbol{\epsilon}, \quad p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \sigma^2\mathbf{I})$$

What about non-linear transformations?

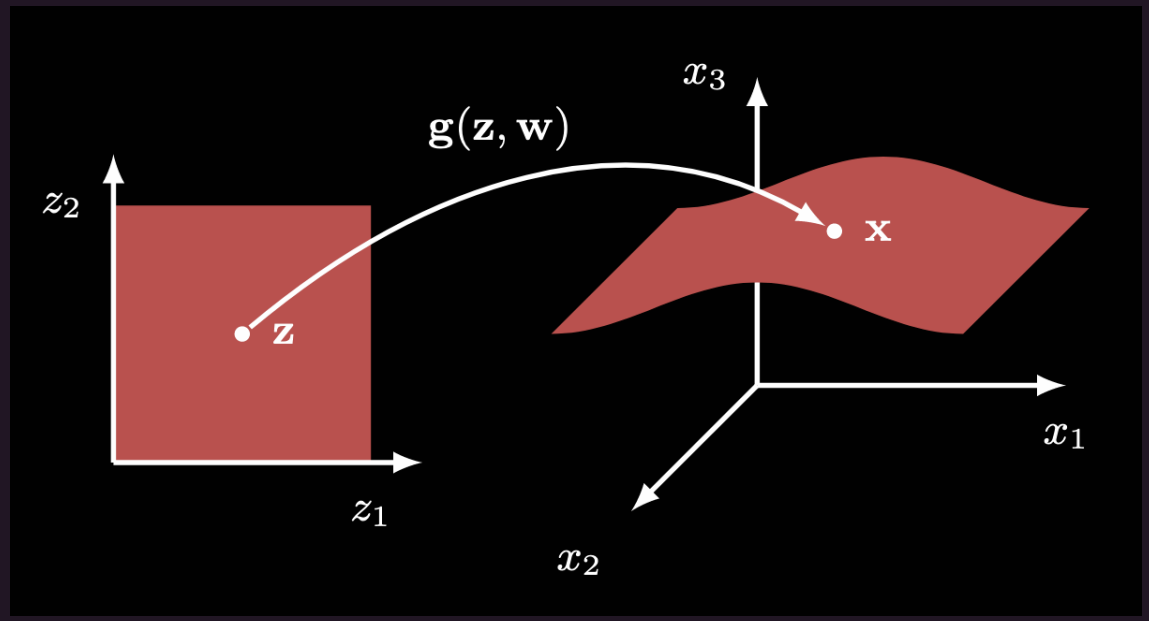
Can we use the flexibility of deep neural networks to model more complex transformations from the latent space to the data space?

$$\mathbf{x} = \mathbf{g}(\mathbf{z}, \mathbf{w})$$

What happens to data not lying exactly on the manifold?

What choices can we make?

$$p(\mathbf{x}|\mathbf{z}, \mathbf{w}) = \mathcal{N}(\mathbf{x}|\mathbf{g}(\mathbf{z}, \mathbf{w}), \sigma^2\mathbf{I})$$

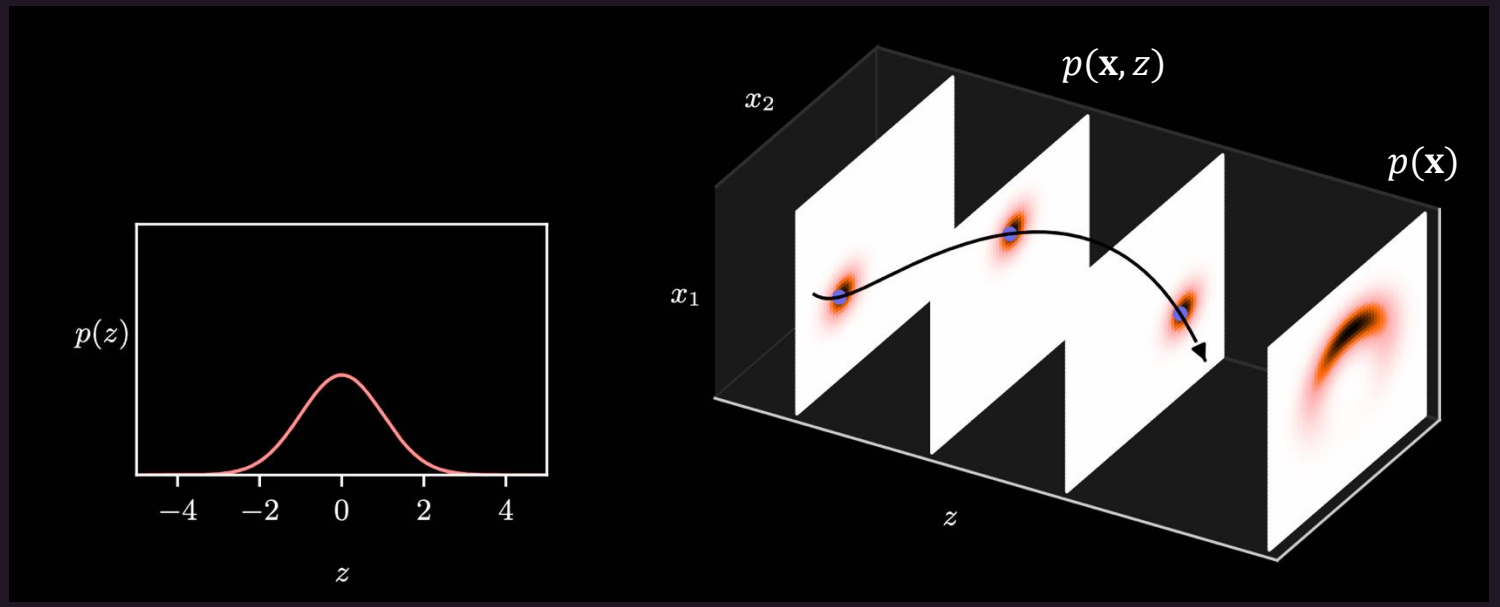


where the neural network has linear output activations, $\mathbf{g} \in \mathbb{R}^D$

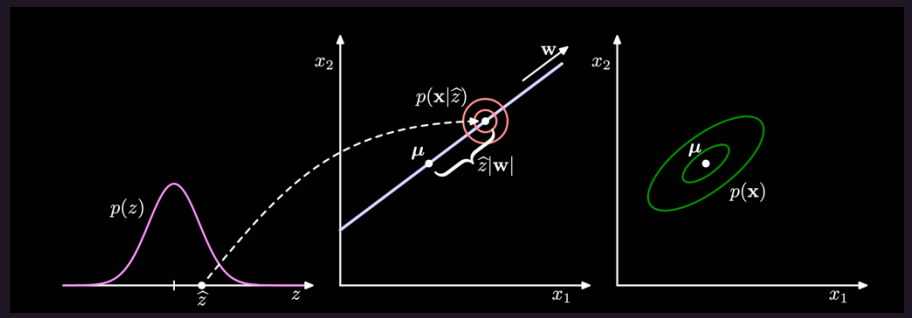
Fig. from "Deep Learning: Foundations and Concepts" – Bishop & Bishop (https://www.bishopbook.com/)

What about non-linear transformations?

$$\mathbf{x} = \mathbf{g}(\mathbf{z}, \mathbf{w}) \qquad p(\mathbf{x}|\mathbf{z}, \mathbf{w}) = \mathcal{N}(\mathbf{x}|\mathbf{g}(\mathbf{z}, \mathbf{w}), \sigma^2\mathbf{I})$$



Recall: PPCA



$$g_1 = \sin(z), g_2 = \cos(z)$$

Figs. from “Deep Learning: Foundations and Concepts” – Bishop & Bishop (https://www.bishopbook.com/)

Let us re-examine the likelihood function

$$p(\mathbf{x}|\mathbf{w}) = \int p(\mathbf{x}|\mathbf{z},\mathbf{w})p(\mathbf{z})d\mathbf{z}, \quad \mathbf{x} = \mathbf{g}(\mathbf{z},\mathbf{w}), \quad p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0},\mathbf{I})$$

Note that $p(\mathbf{x}|\mathbf{z},\mathbf{w})$ is intractable! (the integral cannot be evaluated analytically)

💡 Can we approximate the likelihood? -> the ELBO!

$$\ln p(\mathbf{x}|\mathbf{w}) = \mathcal{L}(q,\mathbf{w}) + \mathrm{KL}(q(\mathbf{z})\|p(\mathbf{z}|\mathbf{x},\mathbf{w}))$$

$$\mathcal{L}(q,\mathbf{w}) = \int q(\mathbf{z}) \ln \left\{ \frac{p(\mathbf{x}|\mathbf{z},\mathbf{w})p(\mathbf{z})}{q(\mathbf{z})} \right\} \mathrm{d}\mathbf{z}$$

$$\mathrm{KL}(q(\mathbf{z})\|p(\mathbf{z}|\mathbf{x},\mathbf{w})) = -\int q(\mathbf{z}) \ln \left\{ \frac{p(\mathbf{z}|\mathbf{x},\mathbf{w})}{q(\mathbf{z})} \right\} \mathrm{d}\mathbf{z}$$

Recap: Deriving the ELBO

$$
\begin{aligned}
\ln p(\mathbf{x}|\mathbf{w}) &= \int q(\mathbf{z}) \ln p(\mathbf{x}|\mathbf{w}) \mathrm{d}\mathbf{z} \\
&= \int q(\mathbf{z}) \ln \left\{ \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} \right\} \mathrm{d}\mathbf{z} \\
&= \int q(\mathbf{z}) \ln \left\{ \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{q(\mathbf{z})} \frac{q(\mathbf{z})}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} \right\} \mathrm{d}\mathbf{z} \\
&= \int q(\mathbf{z}) \ln \left\{ \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{q(\mathbf{z})} \right\} \mathrm{d}\mathbf{z} + \int q(\mathbf{z}) \ln \left\{ \frac{q(\mathbf{z})}{p(\mathbf{z}|\mathbf{x}, \mathbf{w})} \right\} \mathrm{d}\mathbf{z} \\
&= \underbrace{\mathbb{E}_{q(\mathbf{z})} \left[\ln \frac{p(\mathbf{x}, \mathbf{z}|\mathbf{w})}{q(\mathbf{z})} \right]}_{\text{Evidence Lower Bound}} + \underbrace{\mathrm{KL}(q(\mathbf{z}) \| p(\mathbf{z}|\mathbf{x}, \mathbf{w}))}_{\text{Gap to the true log-likelihood}}
\end{aligned}
$$

Back to the likelihood

Consider a set of *iid* training data points from $p(\mathbf{x})$, $\mathbf{X} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\} \in \mathbb{R}^D$

$$\ln p(\mathbf{X}|\mathbf{w}) = \sum_{n=1}^{N} \mathcal{L}_n(q_n, \mathbf{w}) + \sum_{n=1}^{N} \mathrm{KL}(q_n(\mathbf{z}_n) \| p(\mathbf{z}_n|\mathbf{x}_n, \mathbf{w}))$$

where

$$\mathcal{L}_n(q_n, \mathbf{w}) = \int q_n(\mathbf{z}_n) \ln \left\{ \frac{p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w}) p(\mathbf{z}_n)}{q_n(\mathbf{z}_n)} \right\} \mathrm{d}\mathbf{z}_n$$

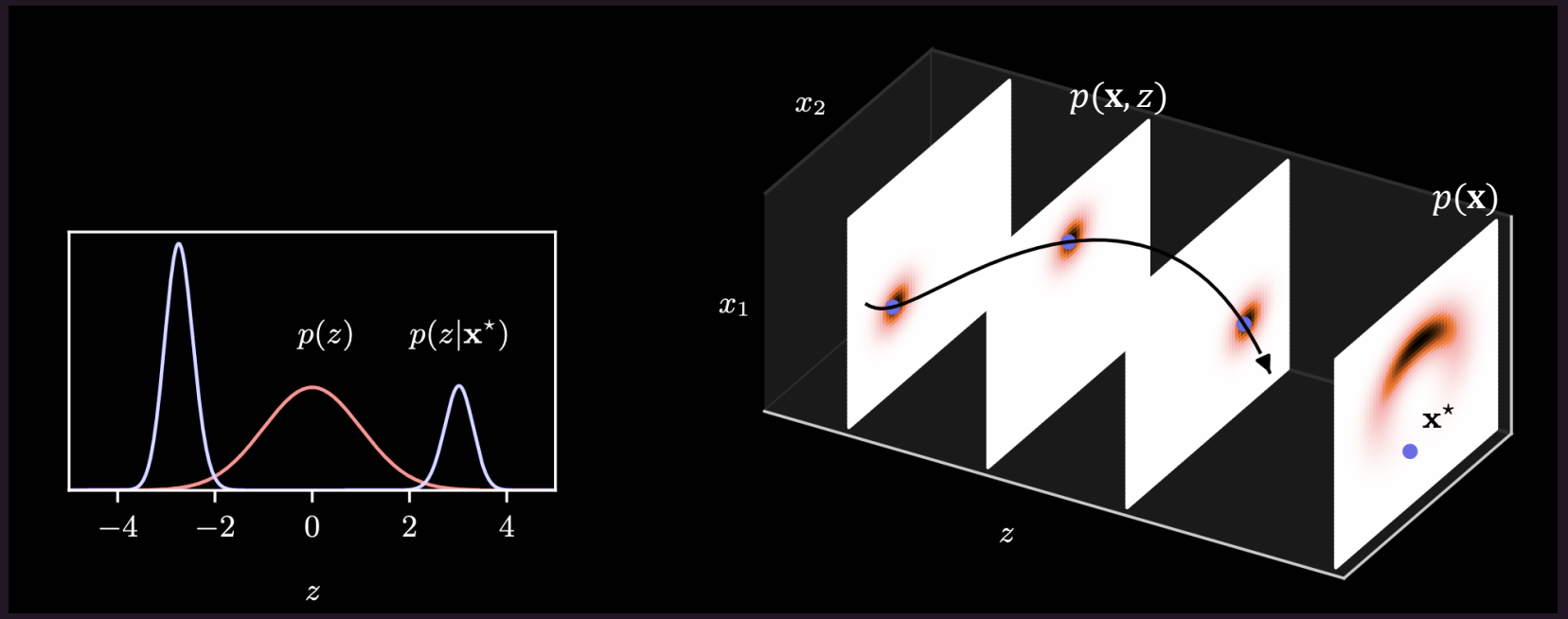
Note that this introduces a separate latent $\mathbf{z}_n \in \mathbb{R}^Q$ corresponding to each $\mathbf{x}_n$

Does this remind you of other models we have seen?

How can we maximize the lower bound?

Maximizing the lower bound

Since we can choose $q_n(\mathbf{z}_n)$, set it to $p(\mathbf{z}_n|\mathbf{x}_n, \mathbf{w})$ to get zero KL



$$p(\mathbf{z}_n|\mathbf{x}_n, \mathbf{w}) = \frac{p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w})\, p(\mathbf{z}_n)}{p(\mathbf{x}_n|\mathbf{w})}$$

<- Intractable!

Amortized variational inference

We could consider a separate parameterized model for each $q_n(\mathbf{z}_n)$; this is expensive for large datasets!

Even if every $q_n(\mathbf{z}_n)$ is a diagonal Gaussian, we have $N \times 2Q$ parameters



Let us train a single network to approximate all these distributions!

We consider an amortized variational posterior $q(\mathbf{z}|\mathbf{x}, \phi)$, where

- ϕ denotes the parameters of the encoder or inference network
- The encoder maps a datapoint to the parameters of the variational distribution

We can consider the variational distribution to be a tractable approximation of the true posterior: $q(\mathbf{z}|\mathbf{x}, \phi) \approx p(\mathbf{z}|\mathbf{x}, \mathbf{w})$

Reparameterization trick

Hang on… The lower bound is still intractable!

$$\mathcal{L}_n(\mathbf{w}, \boldsymbol{\phi}) = \int q(\mathbf{z}_n|\mathbf{x}, \boldsymbol{\phi}) \ln \left\{ \frac{p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w})p(\mathbf{z}_n)}{q(\mathbf{z}_n|\mathbf{x}, \boldsymbol{\phi})} \right\} \mathrm{d}\mathbf{z}_n$$

$$= \int q(\mathbf{z}_n|\mathbf{x}, \boldsymbol{\phi}) \ln p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w}) \mathrm{d}\mathbf{z}_n - \mathrm{KL}(q(\mathbf{z}_n|\mathbf{x}, \boldsymbol{\phi}) \| p(\mathbf{z}_n))$$

Monte-Carlo estimate

$$\simeq \frac{1}{L} \sum_{l=1}^{L} p(\mathbf{x}_n|\mathbf{z}_n^{(l)}, \mathbf{w})$$

$\{\mathbf{z}_n^{(l)}\}$ are drawn from $q(\mathbf{z}_n|\mathbf{x}, \boldsymbol{\phi})$

KL between two Gaussians; can be computed analytically

Reparameterization trick

We can "reparameterize" the random variable $z \sim \mathcal{N}(\mu, \sigma^2)$ using

$$z = \mu + \sigma\epsilon \text{ where } \epsilon \sim \mathcal{N}(0, 1)$$

Note:

1. $z = \mu + \sigma\epsilon$ is differentiable wrt (μ, σ)
2. ϵ does not depend on (μ, σ)

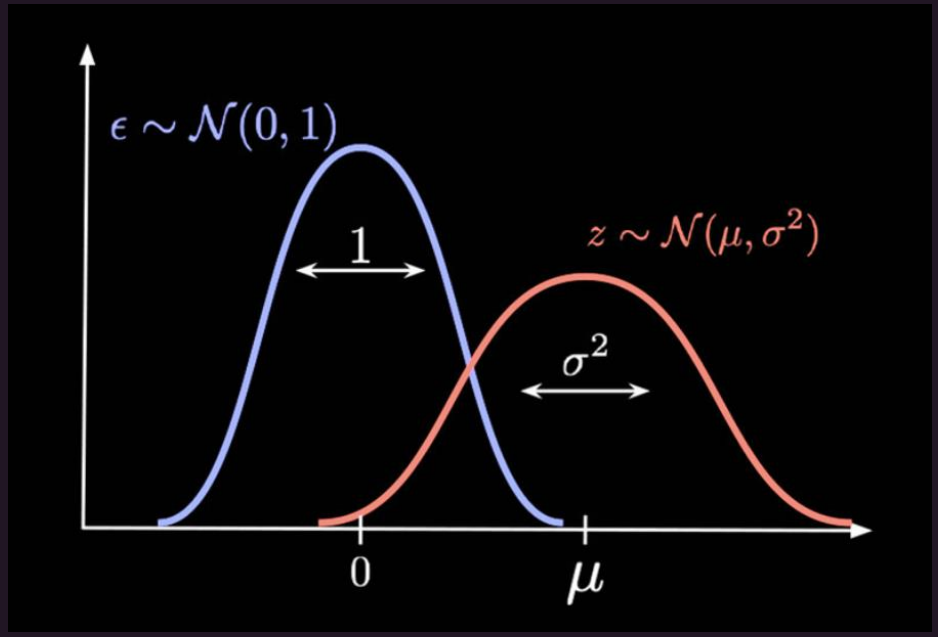
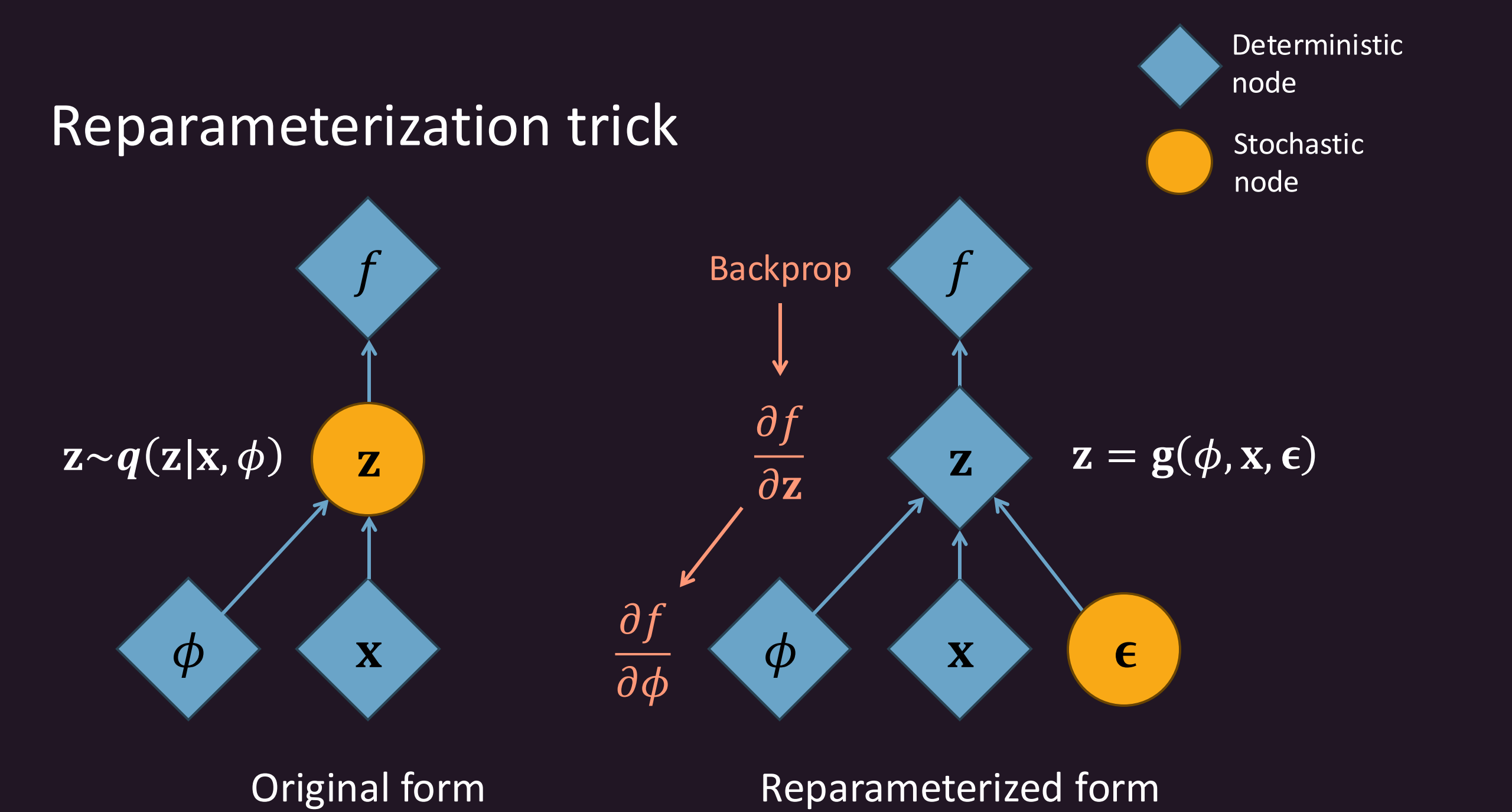


Fig. from "Deep Generative Modeling" – Jakub Tomczak

Reparameterization trick



Original form

Reparameterized form

The Variational Autoencoder (VAE)

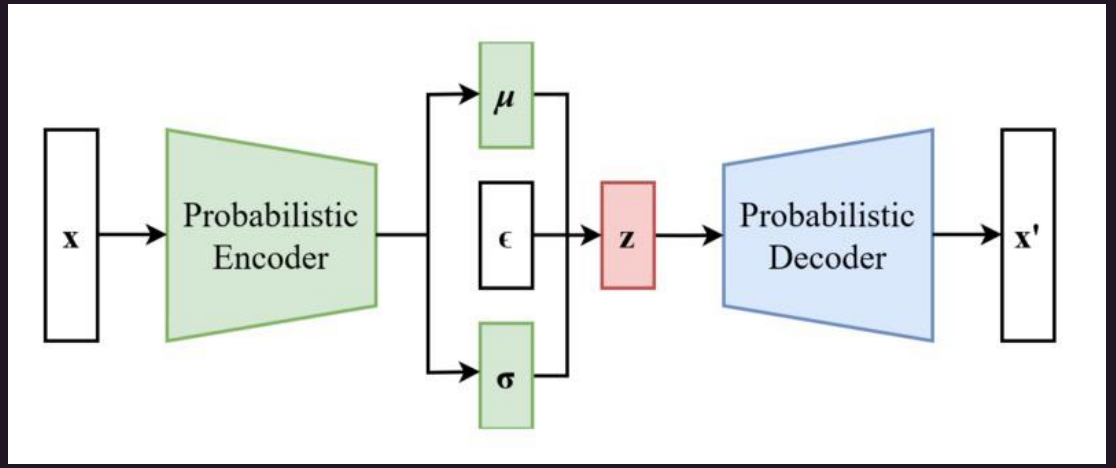
A generative latent variable model consisting of 3 key ideas:

1. Use of the ELBO to approximate the likelihood

$$\log p(\mathbf{x}|\mathbf{w}) \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x},\boldsymbol{\phi})}\left[\log p(\mathbf{x}|\mathbf{z},\mathbf{w})\right] - \mathrm{KL}\left(q(\mathbf{z}|\mathbf{x},\boldsymbol{\phi}) \,\|\, p(\mathbf{z})\right) = \mathcal{L}(\mathbf{x};\mathbf{w},\boldsymbol{\phi})$$

2. Amortized inference in which an encoder approximates the posterior distributions over latents

3. Training is made tractable using the reparameterization trick for computing gradients



Latent representation

$q(\mathbf{z}|\mathbf{x}, \phi) \approx p(\mathbf{z}|\mathbf{x}, \mathbf{w})$ gives a stochastic latent representation of data, e.g. for MNIST:

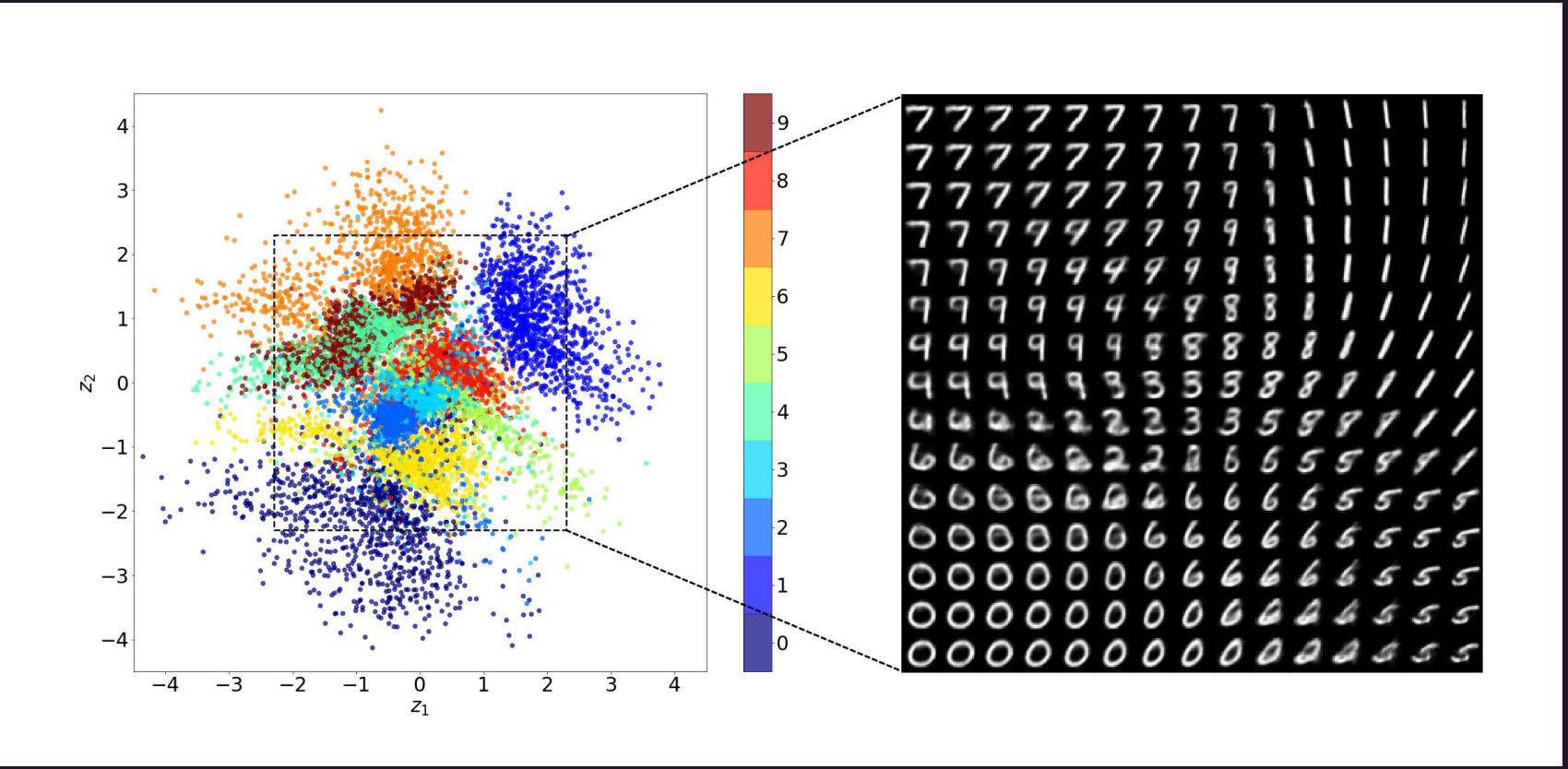


Fig. from https://argmax.ai/blog/vhp-vae/